REPLIC

Reinforcement-Learning-Engineered Protocol for Internet Congestion

A reliable transport over UDP in C with a learned (PPO) congestion controller

Team: Harshavardhan Mallela (24BAI0274), Rohit Arya (24BCE0819), Shloak Sinha (24BDS0378)

Course: Computer Networks, VIT Vellore **Faculty:** Anuradha G. **Date:** 21 October 2026

Abstract. UDP is fast but unreliable, and TCP lives inside the kernel where it cannot be changed. REPLIC rebuilds reliable delivery on UDP in user space: a 13-byte header with CRC-16, sequence numbers, Selective Repeat with SACK, Jacobson-style timeouts and optional XOR forward error correction. Its congestion controller is a small neural network (8-32-32-1) trained with Proximal Policy Optimization, with a fairness term in the reward. On 30 simulated cellular-like paths it moves 1 MB in 6.1 s against 14.9 s for the original fixed-window AIMD sender, and it shares a link fairly with CUBIC (Jain index 0.97).

Key facts. Payload up to 1200 B. Window 256 packets. Controller: cwnd = cwnd x 2^a, a in (-1, 1). Baselines: AIMD, CUBIC, BBR-style, DQN agent.

Demo video: https://youtu.be/dKFTLv3Fsh8 **Source code:** https://github.com/Capsturrrr/rudp

Fig 1. REPLIC architecture

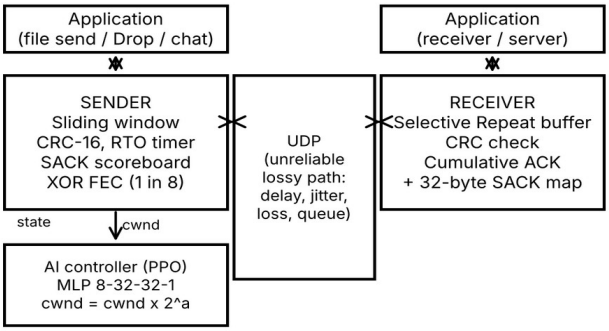


Fig 2. REPLIC packet header

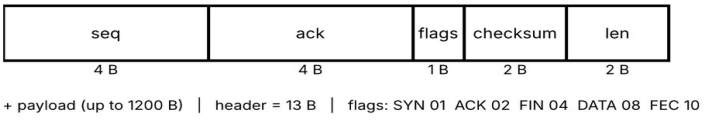
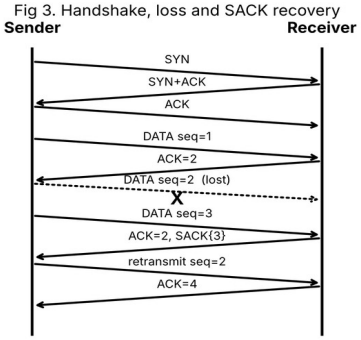


Fig 3. Handshake, loss and SACK recovery



Code 1 of 3: packet format and CRC-16 (src/common.h, src/common.c)

```
#define FLAG_SYN  0x01
#define FLAG_ACK  0x02
#define FLAG_FIN  0x04
#define FLAG_DATA 0x08
#define FLAG_FEC  0x10

typedef struct {          /* header 13 B */
    uint32_t seq_num;
    uint32_t ack_num;
    uint8_t  flags;
    uint16_t checksum;
    uint16_t payload_len;
    uint8_t  payload[1200];
} replic_packet_t;

static uint16_t crc16_byte(uint16_t crc, uint8_t b) {
    crc ^= (uint16_t)b << 8;
    for (int k = 0; k < 8; k++)
        crc = (crc & 0x8000)
            ? (uint16_t)((crc << 1) ^ 0x1021)
            : (uint16_t)(crc << 1);
    return crc;
}

uint16_t replic_checksum(const replic_packet_t *p) {
    uint16_t crc = 0xFFFF;
    for (int s = 24; s >= 0; s -= 8) crc = crc16_byte(crc, p->seq_num >> s);
    for (int s = 24; s >= 0; s -= 8) crc = crc16_byte(crc, p->ack_num >> s);
    crc = crc16_byte(crc, p->flags);
    crc = crc16_byte(crc, p->payload_len >> 8);
    crc = crc16_byte(crc, p->payload_len);
    for (int i = 0; i < p->payload_len; i++)
        crc = crc16_byte(crc, p->payload[i]);
    return crc;      /* checksum field itself is excluded */
}
```

Code 2 of 3: Selective Repeat receiver and SACK acknowledgement (src/server.c)

```
/* SRW = 256 packets. have[] marks buffered packets. */

on_data(seq, payload):
    if seq == expected:                 /* in order */
        deliver(payload)
        expected++
        while have[expected % SRW]:     /* drain buffer */
            deliver(buf[expected % SRW])
            have[expected % SRW] = 0
            expected++
    else if seq > expected and seq - expected < SRW:
        buf[seq % SRW]  = payload       /* out of order */
        have[seq % SRW] = 1
    else:
        ignore                          /* duplicate */

send_ack():
    ack.ack_num     = expected          /* cumulative */
    ack.flags       = FLAG_ACK
    ack.payload_len = SRW / 8           /* 32-byte SACK map */
    for j = 1 to SRW - 1:
        if have[(expected + j) % SRW]:
            ack.payload[j / 8] |= 1 << (j % 8)
    send(ack)

/* bit j of the map = packet (ack + j) has arrived;
   bit 0 is unused because packet ack is the missing one */
```

Code 3 of 3: AI controller, timeout, loss detection (src/smart_cc.h, src/replic_sender.c)

```
/* AI controller: 8 inputs -> 32 -> 32 -> 1 */
sm_forward(x[8]):
    h1 = tanh(W1 * x  + b1)             /* 32 units */
    h2 = tanh(W2 * h1 + b2)             /* 32 units */
    o  = W3 * h2 + b3
    return tanh(o)                      /* a in (-1, 1) */

/* each control interval */
a    = sm_forward(state)
cwnd = cwnd * 2^a                       /* x0.5 to x2 */
cwnd = clamp(cwnd, 2, maxcwnd)

/* retransmission timer: Jacobson + Karn */
rttvar = 0.75 * rttvar + 0.25 * |srtt - rtt|
srtt   = 0.875 * srtt  + 0.125 * rtt
rto    = srtt + 4 * rttvar              /* clamp 60..3000 ms */
/* Karn: never sample RTT from a retransmitted packet */

/* SACK loss detection */
cnt = 0
for s = next - 1 down to base:
    if sacked[s]:  cnt++
    else if cnt >= 3:  retransmit(s)    /* 3 later packets seen */

/* XOR FEC: after every 8 DATA packets send parity
   p1 ^ p2 ^ ... ^ p8; one loss per group is rebuilt
   without a retransmission. */

/* state: RTT gradient, queue-delay trend, queue-delay level,
   loss ratio, inflight/BDP, competing-traffic pressure,
   previous action, log2(cwnd)/7 */
```

```
Complete source code, tests and lab files:  https://github.com/Capsturrrr/rudp
Demo video:  https://youtu.be/dKFTLv3Fsh8
```

2460102-74

Harshavardhan · Mallela

Case Study

Topic: REPLIC · learning-based congestion control over UDP.

1. Application Area

Reliable data transport over lossy, high-latency mobile networks (4G/5G & weak wi-fi) and user space transports such as QUIC [5], where the protocol runs inside the application & can change like without a kernel update. Typical use: uploading a file, sending messages or syncing data from a phone with a poor signal.

2. Background

UDP only delivers packets · no ordering, no retransmissions, no congestion control · TCP adds all three, but lives in the kernel and cannot be modified · its congestion control grows the sending window slowly and halves it on loss [1]. Newer algorithms, such as CUBIC and BBR[4] are better, but each was hand designed for particular network conditions

3. Problem

The classic rule assumes that every lost packet means a full queue. On a cellular link much of the loss is radio noise, so the sender slows down although the link is free, and a short link transfer takes far longer than it should · Retuning the algorithm for every new network is slow expert work.

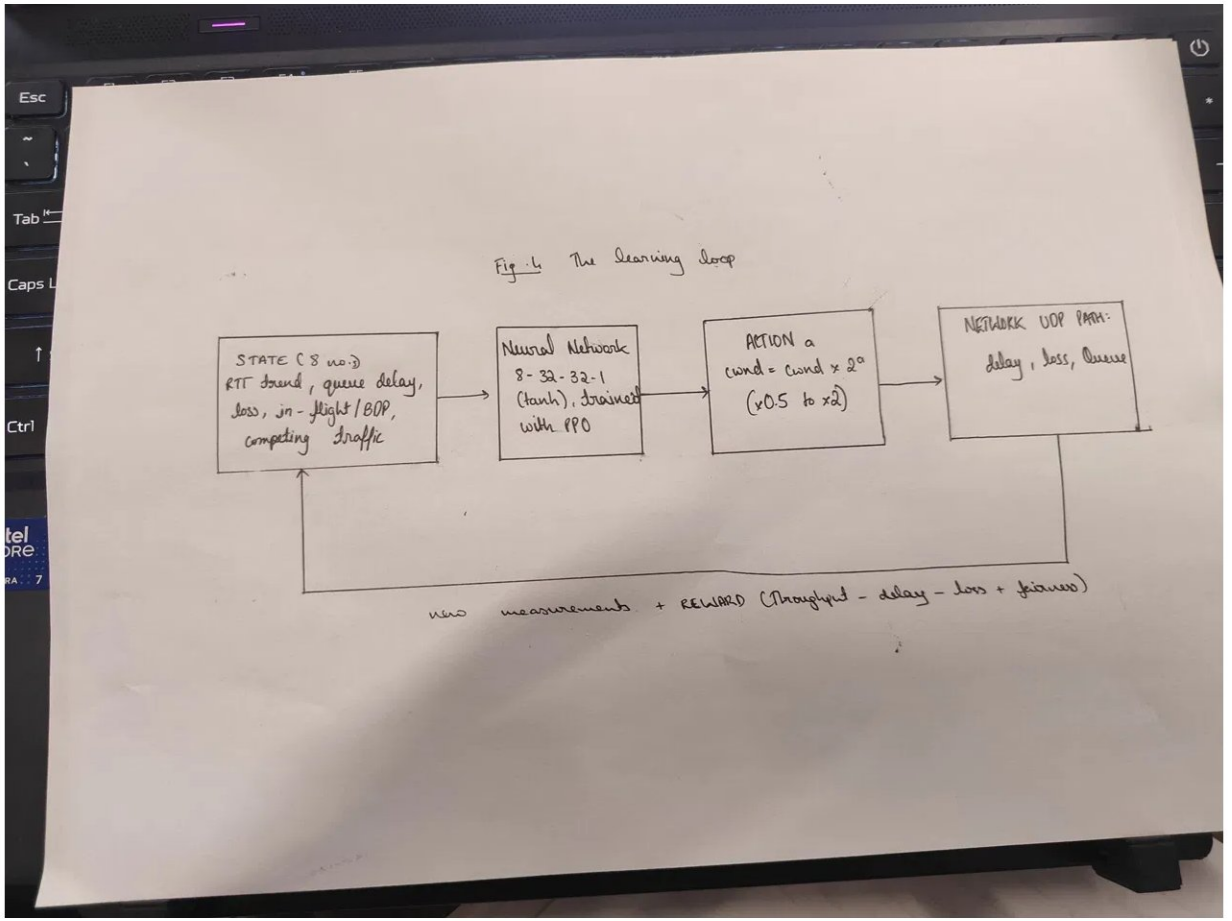
Our Question: Can a very small learned congestion controller, running inside an ordinary C program, do better on such short paths and still share the link fairly.

4. How REPLIC works

(a) "Transport": a UDP protocol in C with a 13-byte header, CRC-16 bit-error detection, sequence numbers, selective repeat with a 256-packet window, and a SACK bitmap so the receiver can say exactly which packets arrived. The sender retransmits when three later packets have been acknowledged or a per-packet timer (with Karn's filter) expires, and an optional XOR parity packet per eight data packets repairs one loss without any retransmission.

(b) ("Controller"): every control interval the sender measures eight numbers (Refer Fig 4) and feeds them to a small neural network. Its output a, between -1 and 1, scales the window by 2^a, so one step can halve or double it.

(c) ("Training") The network is trained in simulation with Proximal Policy Optimization [7]. The reward favours throughput and penalises delay and loss, with a pairwise term so that the flows do not take more than their share. The trained weights are compiled into the C file, so no machine learning library is needed at run time.


Fig. 4 The learning loop
STATE (8 no.)
RTT trend, queue delay, loss, in-flight/BDP, competing traffic
Neural Network
8-32-32-1
(tanh), trained with PPO
ACTION a
cwnd = cwnd × 2^a
(×0.5 to ×2)
NETWORK UDP PATH:
delay, loss, queue
new measurements + REWARD (Throughput − delay − loss + fairness)

5. RESULTS

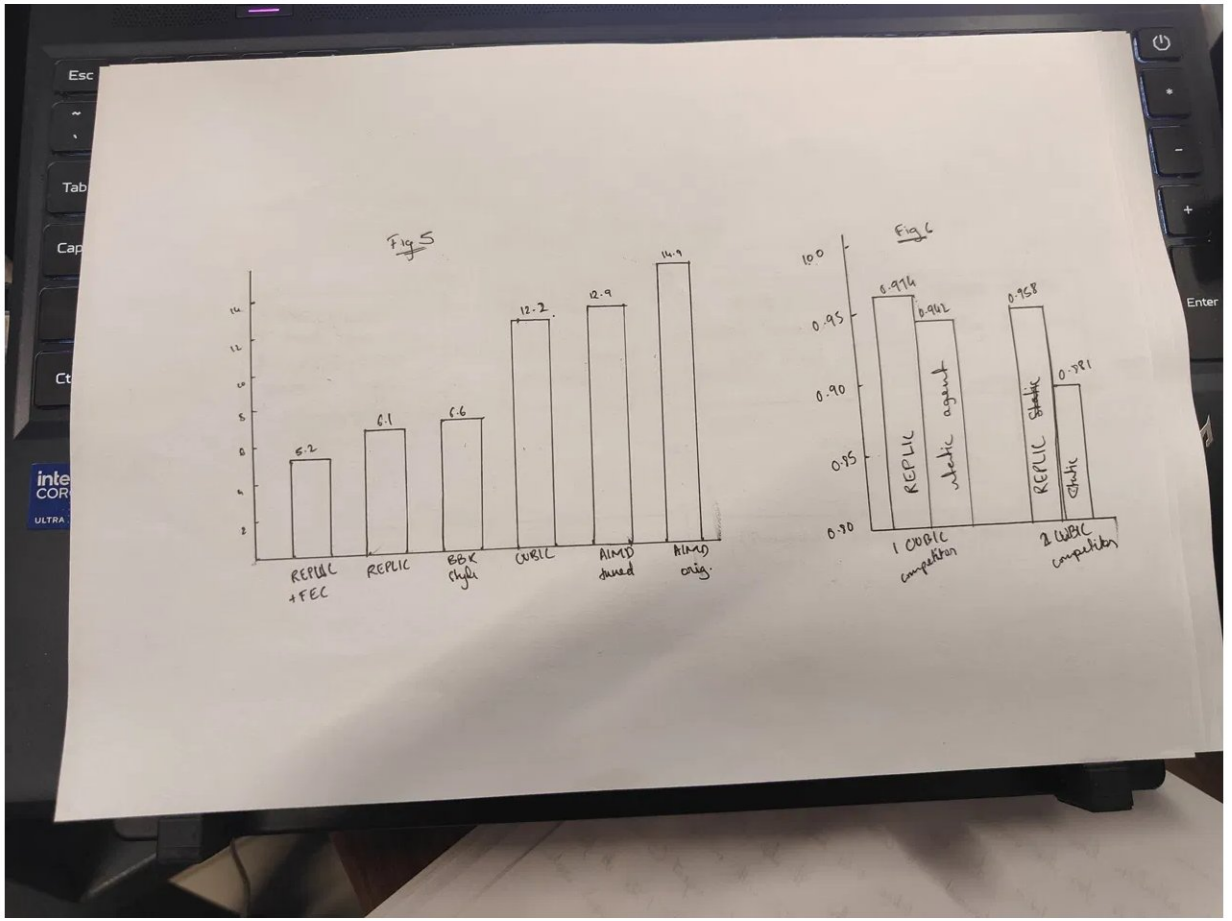
On 30 simulated cellular-like paths a 1 MB transfer took 6.1 s with REPLIC, against 14.9 s for the original fixed-window AIMD sender, 12.9 s for tuned-AI AIMD, 12.5 s for user-space CUBIC and 6.6 s for a BBR-style controller. With forward error correction REPLIC needed 5.2 s. Raised on the same paths, REPLIC takes 0.41 time of the original AIMD sender and its factor on all 30 paths. It also stays fair: against CUBIC competitors the Jain index is 0.974 with one competitor and 0.958 with two, better than the earlier static agent (0.942, 0.881). A network trained without the fairness term took up to 1.72× its fair share, which shows the reward term is doing real work.

6. Future Work

Close the gap to the kernel with faster send path, and train the controller on traces measured on real networks.

7) Conclusion

A controller of about 1300 multiplications per decision, trained by reinforcement learning and written in plain C, beats fixed-rule controllers on lossy long-delay paths and stays fair. A new kind of network needs retraining, not a redesign.


Fig 5
5.2
6.1
6.6
12.2
12.9
14.9
REPLIC +FEC
REPLIC
BBR (tuned)
CUBIC
AIMD tuned
AIMD orig.
Fig 6
1.00
0.95
0.90
0.85
0.80
0.974
0.962
0.958
0.981
REPLIC
REPLIC
1 CUBIC competitor
2 CUBIC competitors

8. Researchers in the area.

Keith Winstein (Stanford University), co-author of Remy [2] and the pantheon testbed [5]. His work showed that congestion control can be generated by optimization & computed jointly on whatever testbeds, which is the approach REPLIC follows with PPO. Other related work: Aurora [3] and Orca [6].

Bibliography

[1] V. Jacobson, "Congestion Avoidance and Control," ACM SIGCOMM, 1988.

[2] K. Winstein, H. Balakrishnan, "TCP ex Machina: Computer-generated congestion control," ICML, 2019.

[3] N. Jay et al., "A Deep Reinforcement learning Perspective on Internet Congestion control," ICML, 2019

[4] N. Cardwell et all., "BBR: Congestion-Based Congestion Control," ACM, Queue 14(5), 2016.

[5] F.Y. Yan et al., "Pantheon: the training ground for internet congestion control ~~for the internet~~," ~~ACM~~ USENIX ATC, 2018.
Research

[6] S. Abdelkrim, C.Y. Yen, H.J. Chao, "Classic Meet Multi Modern: A pragmatic learning-based congestion control for the internet," ACM SIGCOMM, 2020.

[7] J. Schulman et al., "Proximal Policy Optimization Algorithms," arXiV: 1707.06347, 2017.

[8] A. Langley et al., "The QUIC Transport protocol: Design & Internet Internet scale Deployment," ACM SIGCOMM, 2017.